



Procedural trial generation for cognitive assessment: An executive function planning task

Caroline Dakoure¹ · Alexandre Caron¹ · Kajamathy Subramaniam¹ · Delphine Raucher-Chéné^{1,2} · Geneviève Sauvé^{1,2,3} · Élisabeth Thibaut^{4,5} · Katie M. Lavigne^{1,2}

Received: 24 February 2025 / Accepted: 31 March 2026
© The Psychonomic Society, Inc. 2026

Abstract

We introduce an approach for the computational generation of new trials to support the design of cognitive assessments and demonstrate the methodology through a novel executive functioning planning task. Drawing inspiration from procedural content generation (PCG; i.e., computationally creating new content based on a defined set of rules) in gaming, we created a scalable and adaptable method for generating a variety of experimental trials. Our exemplary task, influenced by traditional cognitive measures and digital puzzle games, involves moving colored blocks to form matches and clear the screen, assessing planning. The task development process is detailed from designing mockups through to coding, making the task accessible online, and conducting usability and functional testing. The procedural generation technique, used to automatically generate trials for the task, followed a generate-and-test methodology. The process began with the pseudo-random creation of puzzles (i.e., generate), then a solving algorithm explored the generated configurations and retained only the solvable puzzles (i.e., test). This approach automatically produced many diverse trials for our task. We discuss future directions for PCG in cognitive assessment, including the potential exploration of advanced algorithms to further optimize trial generation. We also provide guidelines for digital cognitive task development and procedural trial generation in cognitive assessment, including incorporating psychometric validation and adaptive difficulty, to facilitate the generation of many high-quality trials for a broad range of applications, including cognitive intervention studies and cognitive ecological momentary assessment.

Keywords Cognition · Digital assessment · Automation · Procedural content generation · Puzzle generation · PsychoPy

Cognitive function refers to “mental processes involved in the acquisition of knowledge, manipulation of information, and reasoning ... [encompassing] perception, memory, learning, attention, decision making, and language abilities” (Kiely, 2014, p. 1078). Assessing cognitive function is important, as cognitive impairments are common in a variety of neurological and psychiatric conditions, significantly

impacting people’s functional abilities and quality of life (Abramovitch et al., 2021; Millan et al., 2012).

Digital cognitive assessment

Digital technologies have improved the assessment of cognitive functioning (e.g., attention and memory; Harvey, 2019) over traditional pen-and-paper measures, and digital cognitive measures have now become the gold standard in research (Chen et al., 2023; Lavigne et al., 2022). Some well-known digital cognitive batteries include the Cambridge Neuropsychological Test Automated Battery (CANTAB; Robbins et al., 1994), the National Institutes of Health (NIH) Toolbox Cognition Battery (Weintraub et al., 2013), and the Cogstate Brief Battery (Maruff et al., 2009). Such digital cognitive tools bring about increased precision (e.g., sub-millisecond timing), automation (e.g., dynamic difficulty, scoring), and standardization (e.g., instructions, stimuli; Lavigne et al.,

✉ Katie M. Lavigne
katie.lavigne@mcgill.ca

¹ Douglas Research Centre, Montreal, QC, Canada

² Department of Psychiatry, McGill University, Montreal, QC, Canada

³ Department of Education and Pedagogy, Université du Québec À Montréal, Montreal, QC, Canada

⁴ School of Psychology, Université Laval, Quebec, QC, Canada

⁵ CERVO Brain Research Centre, Quebec, QC, Canada

2022). They can also be coupled with neuroimaging devices, such as in task-based functional magnetic resonance imaging (fMRI), to explore brain mechanisms underlying cognitive (dys)function. More recent growth in the use and capabilities of personal digital devices (e.g., smartwatches, smartphones, tablets, and personal computers) has led to new frontiers in cognitive assessment (Guimond et al., 2019; Hays et al., 2019; Kilciksiz et al., 2020), including remote assessments in real-world settings (i.e., ecological momentary assessment [EMA]), integration with measures of daily activities (e.g., sleep, exercise), and recruitment of more diverse samples (e.g., from rural communities).

While cognitive measures are typically administered at a single point in time, repeated assessments are often necessary, such as in the context of monitoring disease progression (Llado et al., 2021) and evaluating treatment effects (Vita et al., 2021). Moreover, there is a growing movement to characterize intra-individual cognitive variability (i.e., variations in an individual's cognitive performance over time) to better understand the dynamic nature of cognition in real-world contexts and its associations with various environmental, psychological, and physiological factors within individuals (McKinney et al., 2020). In these contexts, cognitive assessment necessitates the creation of many distinct, yet comparable, high-quality trials, which can require significant effort in both development and implementation. For example, some cognitive batteries can include over a dozen alternate forms to allow for repeated testing (Lavigne et al., 2022). One simple way to address this issue is to design measures that use randomized or pseudo-randomized stimuli; however, this is not always feasible, particularly for cognitive tests involving puzzles (e.g., Tower of London; Shallice, 1982), as they need to remain solvable following a set of rules.

Procedural puzzle generation

Inspiration for designing repeatable cognitive assessments can be drawn from procedural content generation (PCG), a technique from the video game industry. Using computational approaches, PCG algorithmically creates diverse content with minimal user input (Togelius et al., 2011), based on predefined rules and parameters. This method can generate various elements, from individual characters to entire virtual worlds, without manual design of each component (De Kegel & Haahr, 2019; Korn et al., 2017).

De Kegel and Haahr (2019) conducted a review on procedural puzzle generation (PPG), examining 32 methods of puzzle generation across 11 categories of puzzle games, including sliding (e.g., 2048), tile-matching (e.g., *Bejeweled*), maze (e.g., *Pac-Man*), logic (e.g., *Sudoku*), and word (e.g., *Wordscapes*) puzzles. From a cognitive perspective,

these games assess various cognitive processes, and many overlap in content and design with validated neuropsychological tests (Baniqued et al., 2013). For example, maze puzzles are often used to assess executive functioning, such as in the Neuropsychological Assessment Battery (Iverson et al., 2008) and Cogstate Battery (Pietrzak et al., 2009). Digital cognitive measures are, thus, well suited for PCG, which would facilitate repeated assessments while substantially decreasing the burden of task design.

Using PCG in cognitive tasks presents significant advantages as well as challenges (Smith, 2014). One advantage of PCG is that it combines stochastic elements with predefined constraints to generate diverse yet controlled content. This prevents learning effects by introducing variability, while still ensuring that task requirements are met. Learning or practice effects are a major concern in repeated cognitive assessments, particularly with clinical samples, where they can obscure changes over time and responses to treatment (Calamia et al., 2012; Goldberg et al., 2010; Scharfen et al., 2018). While control groups are often employed to correct for learning effects, this may not always be sufficient, given evidence of differential practice effects in healthy and clinical populations (Jutten et al., 2020). PCG provides an exciting opportunity to counteract learning effects in cognitive testing by enabling the rapid development of a large number of similar trials. The scalability of PCG is particularly remarkable, as it can rapidly generate hundreds of trials that can be systematically modified by adjusting algorithm parameters, serving as a valuable tool for adaptive tasks that dynamically regulate difficulty in real time. This automated approach saves time and resources, making large-scale studies and repeated testing more feasible.

A significant drawback of PCG is its high computational demands. Real-time generation during task execution can lead to excessive memory usage and slow performance, potentially disrupting the flow of the task and user experience. Moreover, PCG for cognitive tasks requires both technical skills in algorithm design and a thorough understanding of the task itself to establish clear rules for the algorithm to follow. Additional difficulties include ensuring the quality and relevance of the content generated, as poorly defined parameters can produce trials that are unsuitable or lack meaningful variation, potentially compromising assessment validity. This often requires extensive filtering and adjustment after generation, as well as dedicated psychometric validation procedures for the generated content.

As a case study for procedural generation in cognitive assessment, here we outline the methodology for procedurally generating content for a novel puzzle task assessing the planning subdomain of executive function. Executive functioning is one of the most complex cognitive domains, essential for goal-directed behavior, problem-solving, and adaptive decision-making in daily life (Diamond, 2013). Planning is

commonly assessed in well-established and validated digital measures such as Cogstate (Pietrzak et al., 2009) and CANTAB (Robbins et al., 1994). Moreover, executive functioning lends itself particularly well to puzzle-based tasks, which are well suited for PCG, motivating the development of an executive function task grounded in these principles. We have mitigated several of the potential disadvantages noted above by (1) bringing together a team composed of experts in cognition (KML, DRC, GS, ET), programming (CD, AC), and user experience (UX) design (KS); (2) generating trials “offline,” prior to using them in the task; and (3) ensuring extensive quality control of the procedurally generated outputs and revising the rules, when necessary. This task will be psychometrically validated in clinical and nonclinical populations as part of ongoing work.

Case study: Executive function planning task

Our executive function planning task (<https://doi.org/10.17605/OSF.IO/3PZ74>) was inspired by well-validated cognitive measures commonly used in assessments of planning and executive functioning, including the Tower of London (Shallice, 1982) and its CANTAB variant, Stockings of Cambridge (Robbins et al., 1994). The task was designed as part of a larger Open Cognitive Battery (Lavigne et al., 2025) in collaboration with a team of researchers and clinicians with backgrounds in psychology and neuroscience. The executive function task was also inspired by video games, including Nintendo Puzzle League (Panel de Pon; Intelligent Systems, 1995) and tile-matching games known as Match 3 games. Briefly, our task involves matching three or more blocks of the same color to clear the screen in a specified number of moves. Each puzzle can be solved in the number of moves indicated by moving one block at a time to the left or right using a mouse or touchscreen. As with the traditional measures, such as the Tower of London, several moves must be anticipated in advance to reach the optimal number of moves, thereby capturing the planning subdomain of executive function. We, thus, aimed to create a dynamic, open-source, and adaptable task that would allow for greater flexibility and the ability to generate a virtually unlimited number of unique trials through procedural generation techniques.

The complete project workflow, including task development and procedural generation, is summarized in Fig. 1. The following sections will detail this process in depth.

Task development

Technical requirements

Before task development, a comprehensive list of requirements was established to determine the selection of tools and methodologies to be utilized. This preliminary phase

was critical to ensure that the chosen tools aligned with our specific needs and objectives. By defining these requirements early on, we were able to establish a solid foundation for the subsequent phases of cognitive task development and implementation. This systematic approach facilitated the creation of a task tailored to our goals.

The task needed to meet several key requirements. It had to be interoperable (i.e., able to work with various computer systems, such as iOS or Windows 11) across multiple devices (touchscreen, MRI scanner, laptop) and adaptable for various settings (online, in-person, neuroimaging). A user-friendly interface was essential to minimize cognitive load and make the task accessible to participants with diverse levels of technological proficiency. It needed to be capable of generating numerous repeatable trials while also featuring customizability. Furthermore, it had to integrate seamlessly with data analysis tools, ensure robust data security, and remain cost-effective, allowing for scalable deployment across different research settings without compromising quality. This combination of features ensured that the task was not only scientifically rigorous but also practical and versatile in its application.

Technological tool selection

Figma, a design platform used for creating websites, applications, and products, was used to design mockups, low and high-fidelity prototypes, as well as task sequences (Fig. 2). The task itself was developed using PsychoPy and PsychoJS (Peirce et al., 2019), open-source software developed for designing psychological experiments. PsychoPy enables researchers to design and conduct a wide variety of experiments, without the need for extensive programming knowledge. Together with its online administration platform, Pavlovia, these tools provide the capacity to run tasks online or in person and across multiple devices, from smartphones to fMRI. PsychoPy and PsychoJS outperform many other experiment generators (e.g., NBS Presentation, OpenSesame, Expyriment) in terms of stimulus timing (Bridges et al., 2020), and Pavlovia includes a GitLab code-sharing platform which allows for open-source task sharing and co-development. Additionally, it complies with the EU General Data Protection Regulation (GDPR) and adheres to security standards consistent with North American norms, ensuring data protection and privacy while offering a dependable platform for research. Thus, PsychoPy and PsychoJS were selected as the software for task development due to their flexibility, security features, and status as validated open-source tools. Our task is shared via the Open Science Framework (OSF; <https://doi.org/10.17605/OSF.IO/3PZ74>), which allowed us to generate a digital object identifier (DOI) and include

Comprehensive iterative project workflow

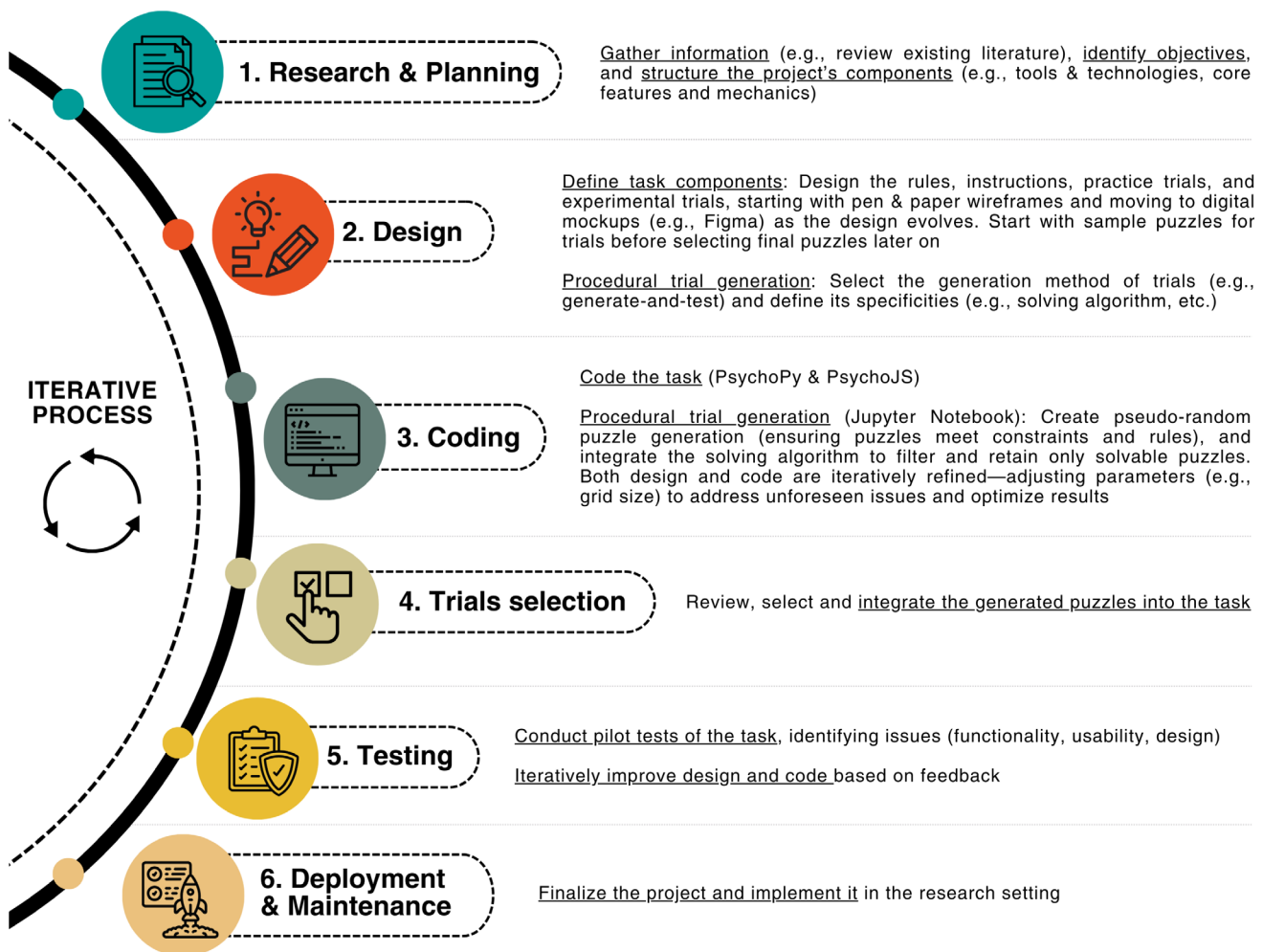


Fig. 1 Comprehensive iterative project workflow

additional wiki documentation. The procedural generation code (<https://doi.org/10.17605/OSF.IO/HNAQ9>) was written in Python and organized as a package called “puzzlegen.” We ran it through Jupyter Notebook (i.e., a web-based tool to run code, show visualizations, and add narrative text for explanations) and executed it using high-performance computing resources provided by the Digital Research Alliance of Canada.

Conceptualization and design

User experience (UX) design

The task design process was guided by core principles of UX design, including consistency, usability, accessibility, user control, contextual awareness, and alignment with the

researcher’s objectives (Caldwell et al., 2008; Norman, 2013). Consistency and usability were prioritized to ensure a low learning curve, making the task intuitive and easy to navigate (Nielsen, 1994). By maintaining consistent design patterns and a clear information architecture, we aimed to minimize the likelihood of user errors and create a straightforward process for error recovery. Furthermore, accessibility considerations (e.g., color scheme, audio and video instructions, touch and mouse capabilities) ensured that the task was usable by individuals with diverse abilities, thereby broadening inclusivity (Persson et al., 2015). User control was emphasized to give participants a sense of confidence and autonomy in their interactions, which can enhance engagement and satisfaction (O'Brien & Toms, 2008). Contextual awareness was also integrated, ensuring that the task remained relevant and meaningful

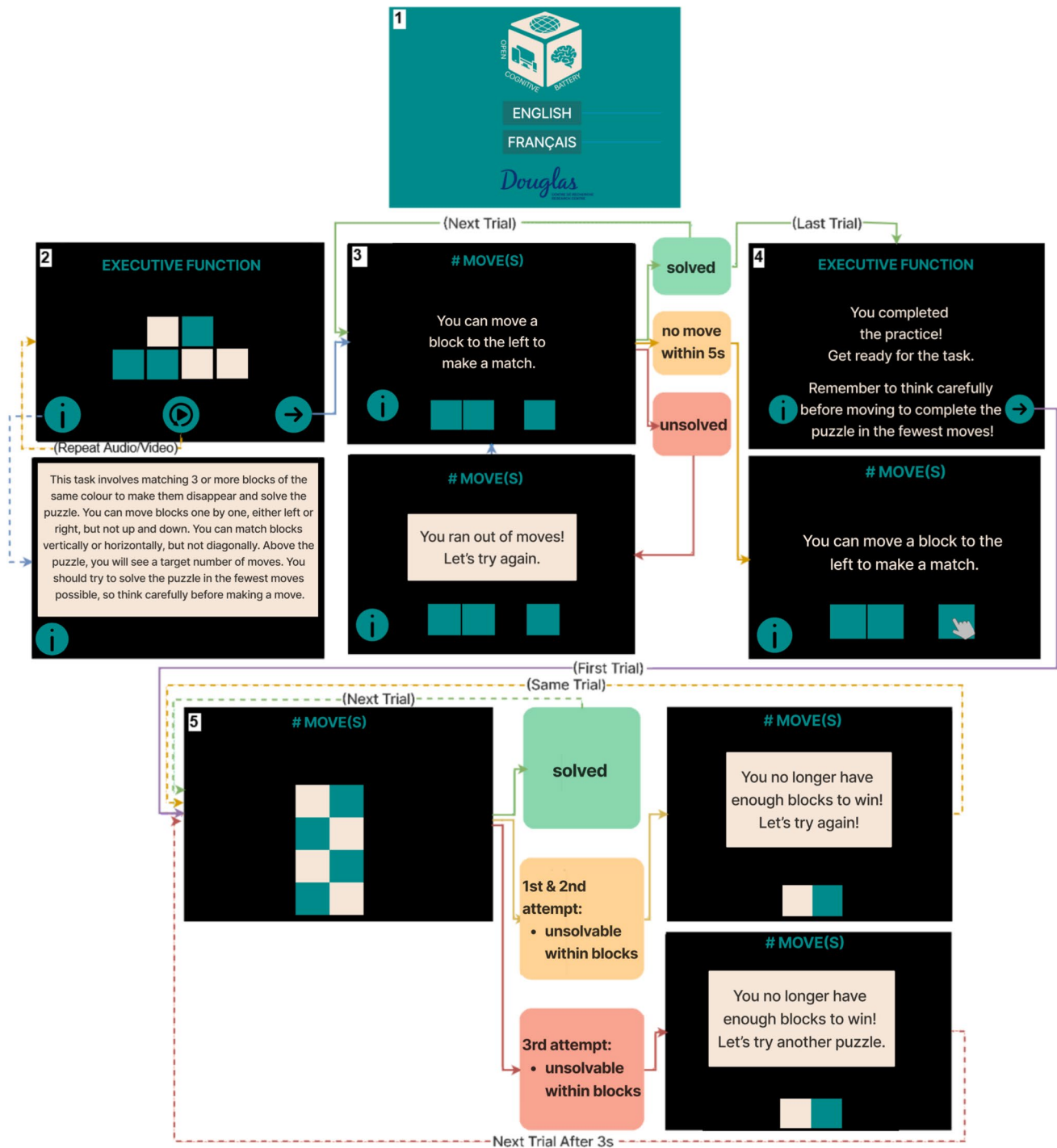


Fig. 2 Overview of the components of the task: language selection, instructional video, interactive practice session, experimental session

within the research framework. By centering the design process around the user's needs and the researcher's goals, the task was crafted to provide an optimal user experience, which will ultimately contribute to the reliability and validity of the task.

Puzzle rules

Our goal was to develop a novel task with rules that efficiently recruit the planning subdomain of executive function by integrating elements from traditional and digital measures. In the following subsection, we will provide an

outline of the puzzle setup and the specific rules governing the task.

Puzzle setup In the initial stage, referred to as the puzzle setup, the game board featured an invisible 2D grid with colored blocks arranged in a specific pattern. Each color had at least three blocks to ensure solvability. Blocks were initialized from the bottom of the grid and could either stand alone or form stacks. Every block, except those in the bottom row, was required to have support beneath it (i.e., another block). To prevent initial matches, no more than two blocks of the same color were placed consecutively, either horizontally or vertically.

Gameplay The gameplay revolved around clearing all blocks from the grid with the optimal number of moves displayed. Players could make as many moves as desired, consistent with similar cognitive tasks (e.g., Tower of London), but were encouraged to aim for the optimal number of moves (i.e., the fewest possible moves). Players could move blocks horizontally only, either shifting into vacant cells (Simple Move) or swapping with neighbors (Exchange Move). The chosen moves encourage strategic thinking and adaptability, as players must consider both immediate outcomes and long-term impacts for their overall strategy. Blocks of the same color vanished when three or more aligned horizontally or vertically (Elimination Rule), which motivates players to form combinations while simultaneously facilitating grid clearance. The task incorporated a gravity mechanism where blocks automatically fell into the empty spaces below when unsupported by other blocks. This rule made it possible to create “match chains,” where additional matches could automatically occur from falling blocks, adding complexity and requiring players to anticipate how their moves would affect the arrangement of blocks and plan accordingly.

Ending condition The puzzle was solved when all blocks were successfully removed from the grid. The puzzle was failed when fewer than three blocks of a specific color remained on the grid.

Procedure

The task includes five components (Fig. 2): a language selection screen (currently in English or French, Fig. 2.1); an instructional video demonstration (Fig. 2.2); an interactive practice session (Fig. 2.3, 2.4); and finally, the experimental session (Fig. 2.5). Players move to subsequent screens at their own pace by clicking on an arrow button.

The *instructional video* includes on-screen button explanations, followed by a video demonstration of the task, with both audio and written instructions (Fig. 2.2).

The *interactive practice* session begins by introducing the game area and available moves. Players can review written instructions via an info button. After 5 s of inactivity, a hand icon appears to suggest a correct move. Failed practice trials require repetition until solved, ensuring comprehension of fundamental task rules before progression. Four practice puzzles (Fig. 3) demonstrate the key task mechanics: horizontal movement, gravity, swapping, and match chains. Players are instructed on how to solve each of these four puzzles. After completing these puzzles, players are reminded of the importance of planning moves before proceeding to the experimental session.

The *experimental session* features puzzles varying in block count, required moves, and color count. Players have unlimited time and moves per puzzle, with an optimal number of moves displayed. We selected 18 puzzles (Table 1) representative of a larger set of procedurally generated trials for initial testing, with larger sets of trials to be psychometrically validated in future work. These 18 puzzles ranged from 1 to 5 required moves, with 6 to 12 blocks and up to three colors. The procedural generation approach, detailed later (see “[Procedural puzzle generation and validation](#)”), enabled us to create a diverse and balanced set of trials for the task. Table 2 shows the outcome measures recorded after each trial.

Task development using PsychoPy and PsychoJS

To develop a task that could be implemented both online and offline across various devices (e.g., smartphones, neuroimaging), we used PsychoPy (offline) and PsychoJS (online), complementary open-source software packages for creating and running experiments. PsychoPy’s drag-and-drop Builder interface facilitates task sequence creation of various stimuli (e.g., images, shapes, sounds, videos) without programming expertise, while also providing a Python-based code component for advanced customization and control. Our complex task required extensive use of the code component to implement the experiment’s logic.

The code component interface in PsychoPy is divided into several tabs, each corresponding to a specific moment in the experiment’s lifecycle. (1) *Begin Experiment*: executes code once at the start, which we used to initialize global variables (i.e., variables used throughout the entire experiment) and set experimental parameters before any trials began. (2) *Begin Routine*: executes at the start of each routine before stimuli presentation, which was used to set up each puzzle (i.e., defining the grid size, block positions and colors, and the optimal number of moves). (3) *Each Frame*: executed continuously during the routine, allowing for real-time updates, dynamic interaction, and immediate response detection. We continuously monitored for mouse/touch presses on blocks. If no blocks were falling, users could move a block

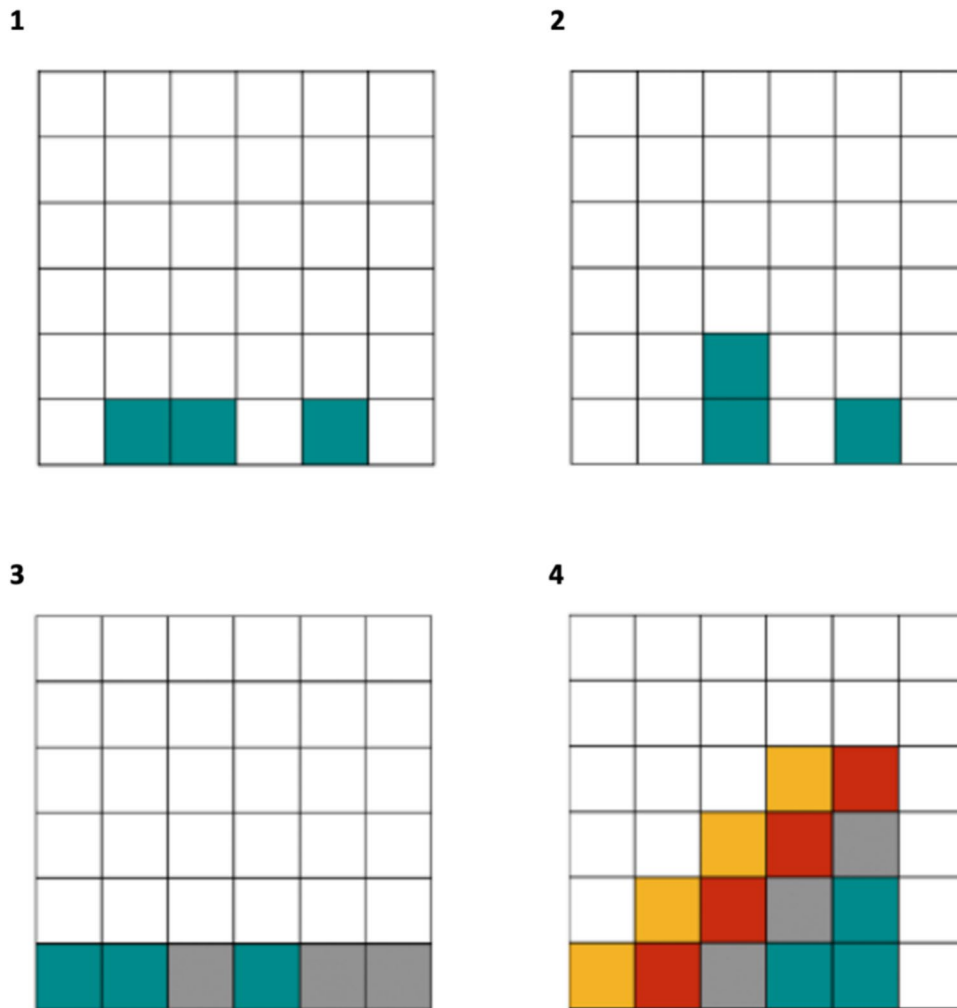


Fig. 3 Overview of the different practice puzzles of the task. Note 1: Demonstration of a simple left move to match three single-colored blocks. 2: Introduction of the gravity mechanism, where moving a block to the right causes it to fall, creating a match. 3: Introduction of the exchange mechanism, where exchanging two blocks in the middle

creates simultaneous matches. 4: Highlight match chains by moving a teal-colored block to the right, triggering a cascade of falling blocks and matches. The aligned blocks are only eliminated once the gravity process is complete

one space left or right by pressing and dragging it. The code checked for empty spaces beneath blocks, causing them to fall (i.e., gravity) until reaching the bottom row or another block. After applying gravity, it checked for horizontal or vertical matches of three or more same-colored blocks in the whole grid, starting from the bottom. After each removal of a set of aligned blocks, it reapplied gravity if needed and rechecked for matches. This process continued until no more matches or falling blocks remained. User interactions were disabled during these checks. Afterward, we set the game state based on the grid's condition: 1 for a win if the grid was empty, 0 to continue if blocks remained, or -1 for a loss if fewer than three blocks of a color remained. We continuously monitored the game state and ended the routine upon win or loss, progressing to the next puzzle or concluding the task as appropriate. (4) *End Routine*: executed after each

trial, used to save data and clear the screen, preparing for the next puzzle or the end of the task. (5) *End Experiment*: executed after all routines, used to close data files and display the ending screen to participants.

After successfully implementing the task offline on laptops or desktops using PsychoPy, we transitioned it to an online environment with PsychoJS, the web counterpart of PsychoPy. This involved exporting the experiment to a web-compatible format (JavaScript). The exported experiment can be hosted on Pavlovia and accessed by participants via a web link. Data collected during the experiment are automatically saved to the server, from where they can be downloaded and analyzed. Although PsychoPy offers automatic translation from Python to JavaScript, differences in syntax and functionality often hindered seamless conversion. We encountered numerous translation issues

Table 1 Experimental puzzle design

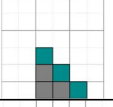
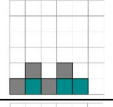
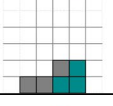
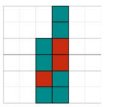
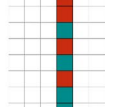
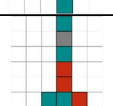
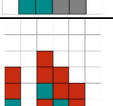
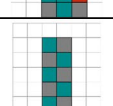
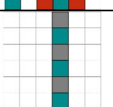
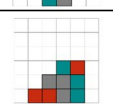
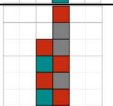
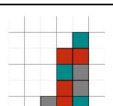
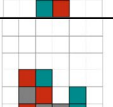
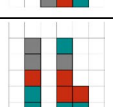
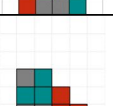
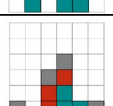
ID	Number of Blocks	Number of Moves	Number of Colours	Figures	ID	Number of Blocks	Number of Moves	Number of Colours	Figures
1	6	1	2		10	7	2	2	
2	6	1	2		11	9	3	3	
3	10	1	2		12	8	4	2	
4	6	2	2		13	11	3	3	
5	12	2	2		14	10	3	3	
6	6	2	2		15	9	3	3	
7	12	2	3		16	10	4	3	
8	9	2	3		17	12	4	3	
9	9	2	3		18	10	5	3	

Table 2 Outcome measures per experimental trial

Name	Description
trAttempts	Number of attempts made to complete the trial
trMoves	Total number of moves executed
trSolved	Whether the trial was solved (true/false)
trRT	The time between the start of the trial and the participant's first action
trSpeed	A list of durations, each representing the time between clicking and releasing a block
trDur	The total duration taken to complete the trial (regardless of success or failure)

Additional measures collected during the practice phase or at the end of the task are not shown here for brevity. The practice phase captures metrics similar to those of the experimental phase, along with additional measures such as the number of times the instructions or video were viewed. At the end of the task, summary statistics are recorded (e.g., totals and means derived from the outcome measures collected during the trials)

despite using the automatic translation feature. Complex or specific Python functionalities, such as advanced functions or libraries, could not convert correctly. Additionally, there were discrepancies between the online version of PsychoJS and the offline version of PsychoPy, with some functionalities available exclusively offline or having limited support in the online environment. Ultimately, users familiar with JavaScript who do not need precise timing (such as in non-neuroimaging experiments) may consider directly coding in that language, as the auto-JS feature may prolong the task creation process.

Functional and usability testing

To ensure cross-device compatibility and multi-language support, we conducted comprehensive testing with 12 members of our research group, representing diverse backgrounds (research assistants, project coordinators, developers, undergraduate students, and UX experts) and using various devices and browsers. A structured sheet, available on our OSF repository (<https://doi.org/10.17605/OSF.IO/3PZ74>), was employed to document issues, categorizing them as functionality, usability, or design-related. Reviewers provided detailed information about their testing environment and problems encountered (e.g., device used, operating system, screen size, browser, semantic version of the task, description of the problem and its location).

Functionality issues concerned the behavior of the experiment, arising when something did not function as expected or when an essential feature was missing or poorly implemented. *Usability* issues pertained to ease of use and user experience, reflecting difficulties in navigation or user-friendliness of the interface. *Design* issues focused on visual and aesthetic aspects, noting inconsistencies or distractions in visual elements. Space was allocated to note whether the issue was replicated by other users, propose actions for resolution, obtain approval from the main developer, prioritize changes, and track the status of changes (Open, In Progress, Resolved, Partially Resolved, or N/A). Finally, the version of the task where these issues were resolved was also tracked.

Having detailed the conceptualization, design, and development of our task, as well as the internal testing that helped us ensure the functionality of the task and improve the user experience, we now turn to another crucial aspect of our project that was conducted concurrently: the automated generation of puzzles. While we were refining the task's interface and functionality, we also focused on implementing an automatic puzzle generation system. The following section outlines this process, explaining how we implemented an algorithm to generate a wide range of puzzles with controlled features, ensuring a scalable and repeatable task.

Procedural puzzle generation and validation

Design

Based on the review conducted by De Kegel and Haahr (2019), our executive functioning task primarily aligned with the tile-matching (Match 3) category, but also incorporated elements from Sokoban, sliding puzzles, and logic puzzles, while distinguishing itself by not continuously generating new objects like popular puzzles such as Tetris or Bejeweled in the same genre. The predominant PCG approach in the tile puzzle category involves a generate-and-test methodology (De Kegel & Haahr, 2019), which consists of two main phases: (1) the puzzle layout is pseudo-randomly generated; and (2) a check is performed to confirm the puzzle is solvable. If the generated puzzle proves unsolvable, it is discarded, and another puzzle is generated.

To follow the generate-and-test approach, we selected a solver that was capable of completing the generated puzzles with a minimum number of moves (optimal solution) while being easy to implement. Computational time complexity (i.e., duration to run the program) was not considered critical, as our PCG script operated asynchronously (i.e., outside the implementation of the task itself). The intention was to design an algorithm to generate a large set of trials offline for future psychometric validation.

Development

In this section, the main steps from generating blocks on the grid to finding a solution are detailed. To facilitate automated puzzle generation, we developed a Python package called “puzzlegen,” which integrates all functionalities into a single tool. It is accessible via local installation or through a user-friendly Jupyter Notebook (<https://doi.org/10.17605/OSF.IO/HNAQ9>).

Generation algorithm

The generation function (Fig. 4) began by looping over a defined number of blocks and colors, randomly assigning a color to each block, and ensuring a minimum of three blocks per color (Fig. 4.1). Block placement started with the first block in one of the two center columns to avoid visual field bias (Fig. 4.2). Subsequent blocks were either stacked or placed in the bottom row based on a stacking probability parameter. Stacking occurred on the highest block in a randomly selected column, while bottom row placement adhered to a maximum gap parameter from neighboring blocks. Before finalizing the placement of the block on the grid, the algorithm checked for three-block alignments (Fig. 4.3). If such alignments were detected, the random

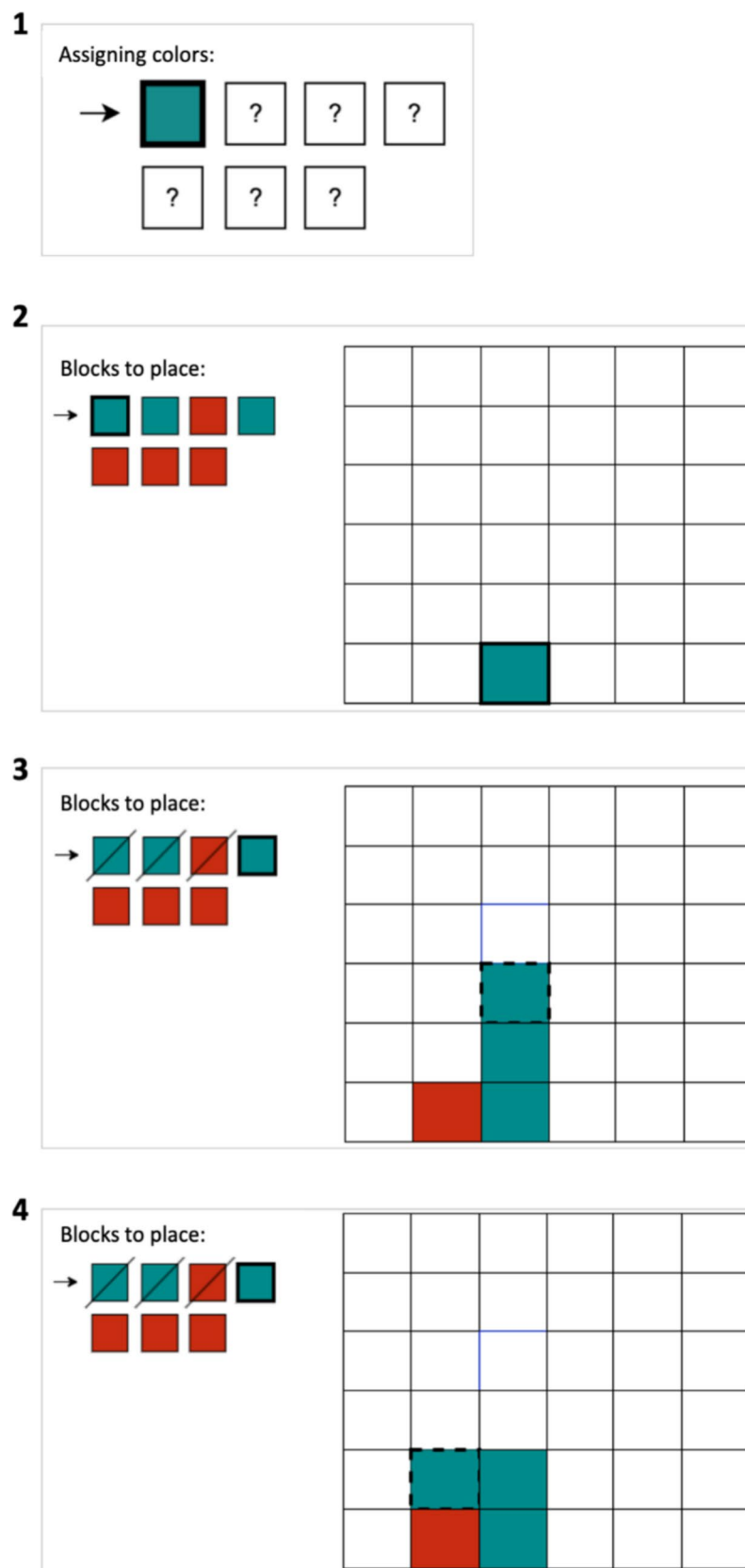


Fig. 4 Puzzle generation: Initialization algorithm. Note. 1: Random color assignment ensuring at least three blocks of each color. 2: Initial block placement in central column on the grid floor. 3: Subsequent block placement (stacked, invalid). 4: Subsequent block placement (stacked, valid)

placement process was reattempted, maintaining adherence to stacking and gap parameters, until three matching blocks were no longer aligned (Fig. 4.4).

Automatic solver

After generating a puzzle that followed task rules, we implemented a breadth-first search (BFS) algorithm (Bundy & Wallen, 1984) to find a solution. BFS is widely used for pathfinding and shortest path determination in various applications, including games, GPS navigation, and web crawlers. BFS can be understood through the analogy of searching for a specific item hidden somewhere in a multistory building. Each “room” in this analogy represents a distinct configuration of the puzzle—that is, a particular arrangement of the blocks (i.e., their positions and colors) resulting from a sequence of moves. Rooms on the first floor correspond to configurations reached after a single move from the initial state; those on the second floor are reached after two moves, and so on.

BFS systematically explores all rooms on the first floor before proceeding to the second, and so forth. This approach guarantees that the shortest route to the target is identified, without prematurely committing to deeper but potentially inefficient paths. Once BFS locates the room containing the item—in our case, the final configuration in which the puzzle grid is empty because all blocks have been cleared—it stops and reconstructs the shortest path from the initial state to the solution.

Our algorithm began at the initial generated configuration, exploring blocks in the bottom row from left to right, simulating movements to the left, followed by movements to the right, and then exchanges to the left and right (see Fig. 5). Upon exploration, the resulting configurations were saved after each move and set aside for future exploration. This process continued until all blocks in the initial configuration had been examined. Only after evaluating all configurations reachable in one move did the algorithm proceed to configurations reachable in two moves, and so on. This approach ensured the identification of the shortest solution path, although alternative solutions may exist with the same number of moves. Our algorithm explored all configurations until a solution was found, efficiently identifying the minimum-move solution while avoiding unnecessary and resource-intensive exploration. The entire BFS exploration process is represented as a graph in Fig. 6.

Our BFS method utilized a specific structure to organize and process information called a queue, which operates similarly to a line of customers waiting at a store: The first individual to join the line is the first to be served. Each puzzle configuration (or “state”) is added to the back of the line and taken from the front, ensuring they are explored in the exact order they are discovered. This method is efficient for managing a large number of configurations without overloading memory.

Within this queue, we maintained detailed records using lists: one list tracked the sequence of puzzle configurations, showing the blocks’ positions at each step, and another list recorded the types of moves made to reach those configurations. By keeping this history, we could calculate how many moves were needed to solve the puzzle. Although each puzzle configuration inherently contains this historical information, to streamline our explanations, we refer primarily to the overall puzzle configuration. Figure 7 illustrates the queue’s contents during the BFS search process.

For a concise summary of all the steps explained above, the pseudocode of our BFS algorithm is provided in Fig. 8.

BFS considerations

One limitation of BFS is that it explores all possible options one by one (i.e., in a brute force manner), without using any shortcuts based on prior knowledge about the task (e.g., recognizing favorable block dispositions or recurring patterns). This exhaustive approach can lead to longer execution times, as BFS does not make use of insights that could streamline the search process. This is in contrast to more advanced algorithms, which use game-based strategies (e.g., distance between blocks of the same color) to more efficiently find solutions, which improves performance; however, they may not find the solution that involves the fewest number of moves, which was essential to ensure this task was conceptually similar to well-validated planning tasks.

BFS also does not identify all possible winning move patterns but finds the first pattern within the smallest number of moves. This is efficient but non-exhaustive. For example, in Fig. 6.3, configuration J would also lead to a puzzle solution in the same number of moves as configuration I. The BFS solver would not identify this solution, as it terminates on the previous configuration. Thus, when multiple solutions using the same number of moves are possible, the algorithm will prefer solutions where blocks move to the left, simply due to the order in which we designed the solver to check for solutions. Importantly, this would not affect participants’ scores during task administration, where both solutions would be interpreted as correct.

Challenges in generation and improvements to constraints

To generate a large variety of puzzles, the puzzles were produced in batches where the algorithm looped through the blocks, colors, and grid size to generate puzzles. We generated multiple batches with different parameters to select those that would be ideal for the task. Examples of generated batches are openly available on our OSF repository (<https://doi.org/10.17605/OSF.IO/HNAQ9>).

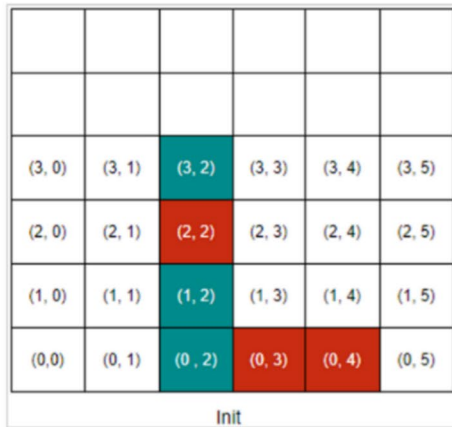
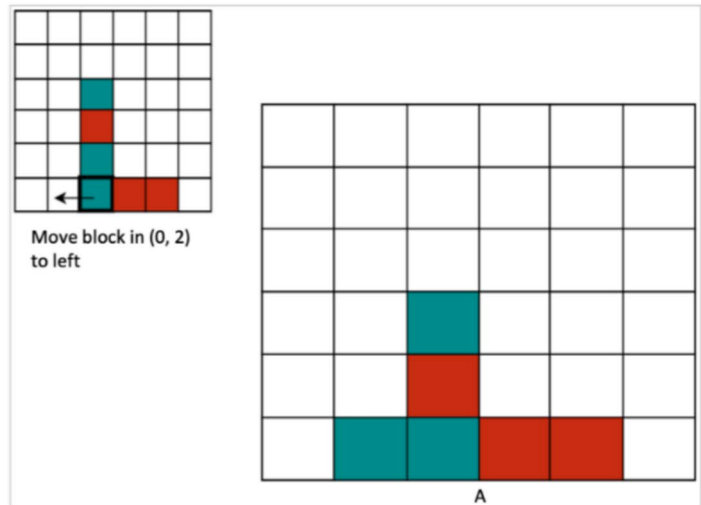
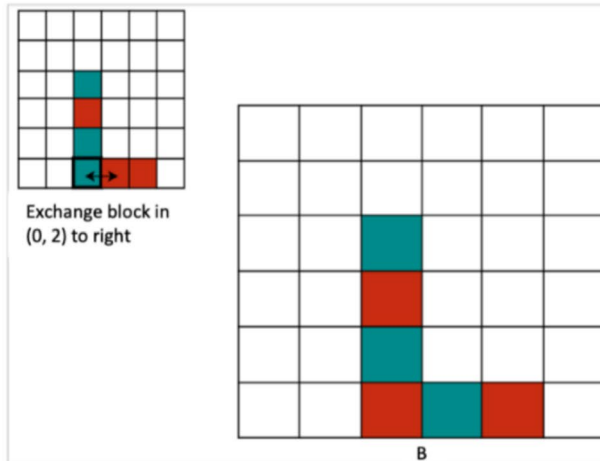
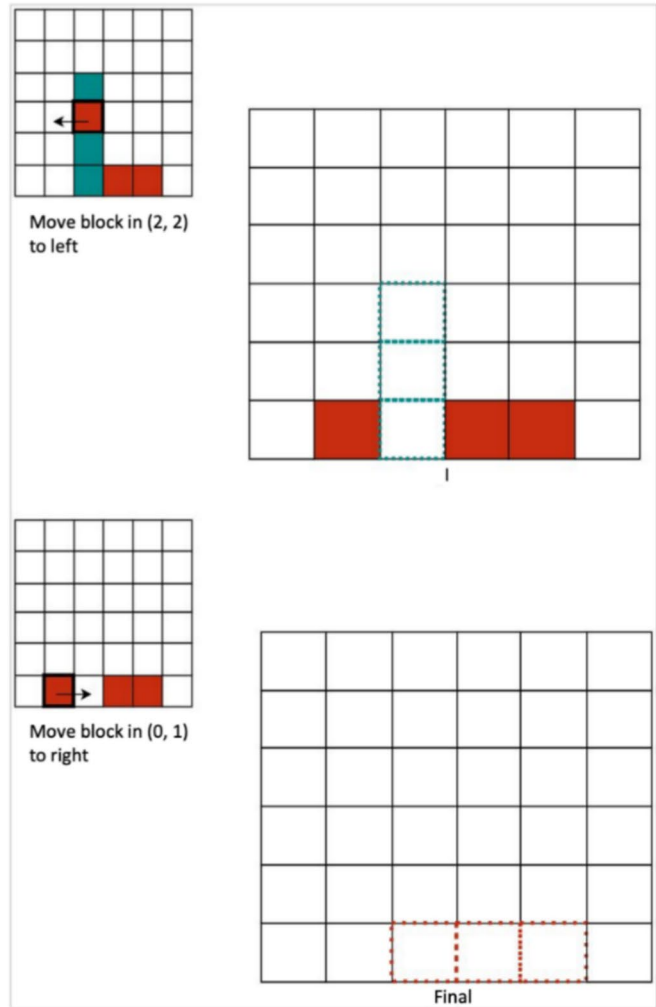
1**2****3****4**

Fig. 5 Example of steps performed by the BFS algorithm. *Note.* 1: Displays the initial configuration, Init. The first block explored was located at (0, 2). From this position, the algorithm assessed possible moves, including moving left or exchanging with the block on the right. 2: Illustrates the subsequent configuration obtained after moving the block at (0, 2) to the left. 3: Depicts another configuration derived from Init after exchanging the block at (0, 2) with the block to its right. 4: Shows the final configuration achieved in several steps reached by moving the block at (2,2) to the left, which caused the elimination of the teal blocks (resulting in the I configuration). Subsequently, the block at (0,1) was moved to the right to match the red blocks, resulting in the Final configuration

During initial grid generation attempts, we identified several issues that were addressed by implementing additional constraints. Grid size was found to be a critical factor influencing successful block generation and simulation efficiency. Overly small grids (e.g., 4×4 with seven teal blocks) risked generation failure due to insufficient space to comply with the initialization constraints, while excessively large grids (e.g., 16×16 with five teal blocks) led to prolonged generation and solver simulation times. A grid height that was too small could also hinder block stacking, while larger grids increased the likelihood of numerous small stacks and widely spaced blocks, potentially resulting in less desirable puzzles requiring many moves to solve. We, thus, established grid dimensions with a minimum size of 4×4 to ensure the puzzle's solvability, while the maximum size was set at 12×6 to optimize computation time.

The algorithm occasionally struggled to generate puzzles solvable in fewer than five moves (the solver was limited to a maximum of five moves to remain consistent with the original Tower of London design), particularly with high block and color counts. In such cases, the generation process was restarted up to five times. If no solvable puzzle was found after these attempts—indicating that for a specific set of parameters (e.g., 3 colors and 11 blocks), no puzzles were found in this batch—the algorithm advanced to the next set of parameters and attempted new puzzle generation.

To mitigate these issues, we reduced the grid size when there were many blocks and colors, and we added two additional constraints to further optimize the pseudo-random placement: a gap parameter and a stacking probability. The *gap parameter* defined the distance a new block could be placed from existing blocks (in our case, one empty cell between the new block and an existing block). The *stacking probability parameter* influenced the shape of the configurations and defined the likelihood that a block would be placed on top of an existing block, versus on the bottommost row of the grid. Specifically, a 0% stacking probability led to stacking only when the bottom row was filled, while a 100% probability resulted in blocks stacked in the same column until height limits were reached, then initiating a new stacking column. Varying this parameter (0%, 25%, 70%, 75%, 100%) allowed us to generate diverse puzzle sets.

Visualizations

Figure 9 is an example of puzzles obtained during batch generation. This batch was designed with the following parameters: containing 3 to 12 blocks, 1 to 3 colors, solvable in a maximum of 5 moves, grid sizes ranging from 4×4 to 12×6 , up to 1 gap between blocks, and a 70% stacking probability. The generation algorithm then looped through the blocks and colors to generate puzzles, starting with 3 blocks and 1 color, followed by 4 blocks and 1 color, and so on, until attempting to generate a puzzle with 12 blocks and 3 colors. Note that, for readability, this example does not include all puzzles obtained.

Given its random nature, the automatic generation frequently yielded uninteresting puzzle configurations (e.g., single color with numerous blocks, overly complex puzzles) that were discarded. The third puzzle depicted in Fig. 9 exemplifies an uninteresting puzzle configuration characterized by eight blocks of the same color. Figure 10 shows the results from the batch generation, which is detailed in Table 3. In total, 21 puzzles were generated in this batch without any failures. Table 3 provides a detailed breakdown of the 21 puzzles, including the number of blocks per puzzle, the number of colors used, and the minimum number of moves required to solve each puzzle. Table 4 demonstrates a batch generation with parameters that resulted in puzzles unsolvable in a maximum of five moves.

Discussion

Here, we presented the development and implementation of a procedural trial generation system for cognitive assessment, demonstrated with a novel executive function planning task. Procedural generation has been used extensively in gaming and the computer science literature and has immense potential for developing scalable and diverse trials for repeated digital cognitive assessment. This technique is adaptable for a wide range of cognitive measures involving the strategic presentation of stimuli over multiple trials. To guide future work, below we provide several guidelines for digital cognitive task development and procedural trial generation.

Digital cognitive task development guidelines

Favoring interdisciplinary collaboration

Developing digital cognitive measures and procedurally generating trials require interdisciplinary expertise. We brought together a team of experts in cognition, computer science, and user experience/design, which helped to ensure that our task was technologically innovative and engaging, while maintaining face validity with traditional measures.

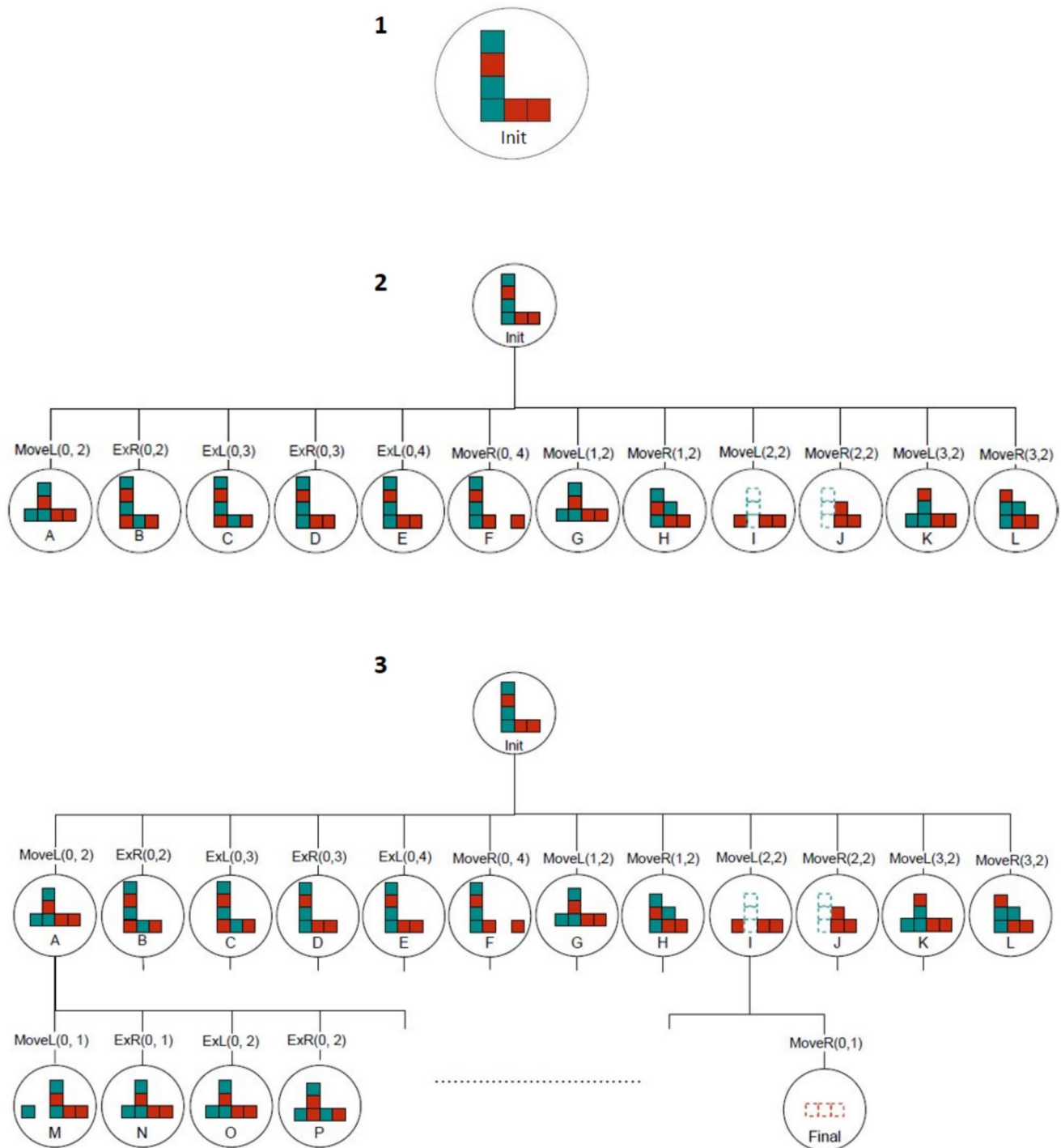


Fig. 6 Graph representing the different configurations obtained after each move performed by the BFS algorithm at different steps. Note. For clarity, not all *moves* and *configurations* are depicted. Each node (i.e., circle) captures both the puzzle configuration and the history of moves and configurations leading to it. 1: Represents Level 0, which

is the initial configuration to be explored, Init. 2: Illustrates Level 1, consisting of the configurations obtained after one move of any block from Level 0. 3: Depicts Level 2, which includes the configurations derived from Level 1, leading to the final configuration, Final

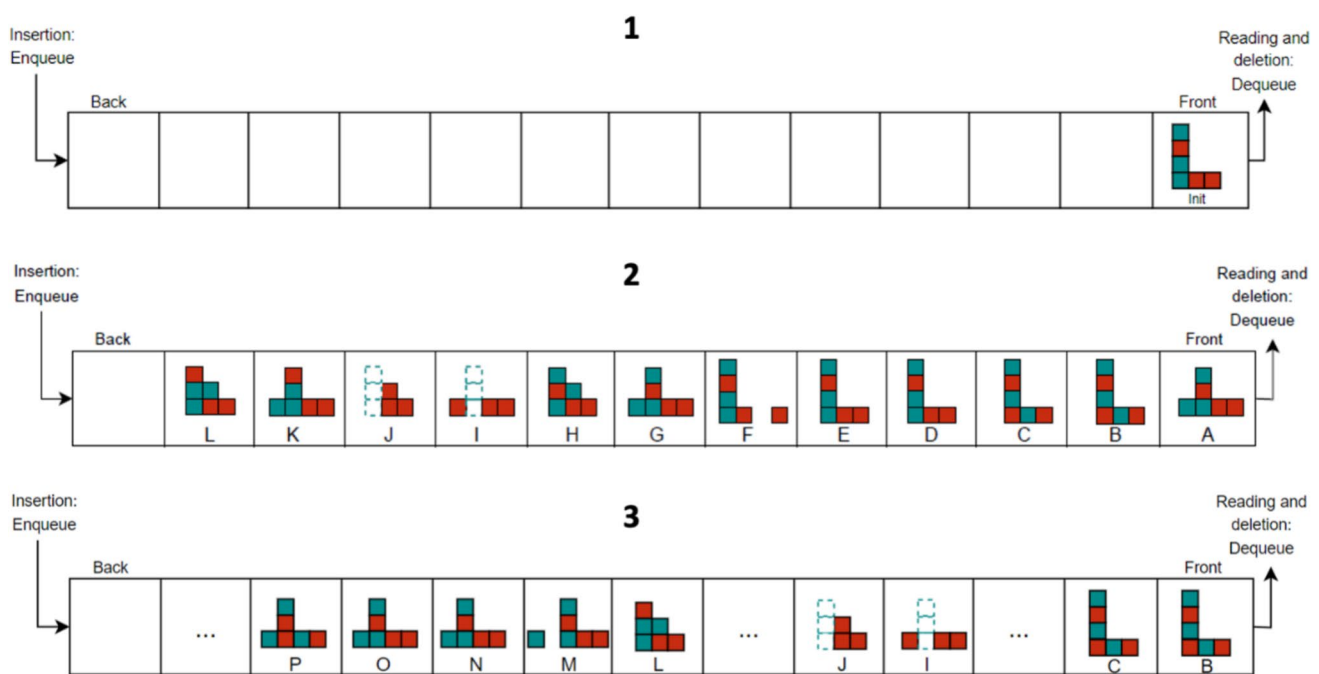


Fig. 7 BFS queue status after adding different configurations at different steps. *Note.* 1: Illustrates what the queue stored in memory at the initial state. It contained the initial configuration with all blocks and their initial positions. 2: Depicts what the queue stored in memory after all new configurations obtainable in one move from 1 were

added. 3: Displays what the queue stored in memory after the A configuration had been explored (new resulting configurations start from L). The subsequent steps are not represented, but when configuration I reaches the front of the queue, a solution is found during its exploration and the algorithm terminates

Designing mockups

Before starting to code the task, comprehensive mockups should be designed that illustrate the visual and interactive aspects of the task. These mockups should include all interactions, instructions, and stimuli (see Fig. 2), clearly describing the actions that should occur after specific triggers. This will help with the conceptualization of the task and the reflections that precede the coding phase. Knowing what the task should look like facilitates the coding process.

Incorporating elements that will increase impact

Developing digital cognitive tasks is labor-intensive; designing tasks that can be shared with the broader community will increase their reusability. The task should be easily adaptable to meet the needs of diverse researchers and participants (e.g., language, ability, setting). Our task is bilingual (English and French), with a language selection component to facilitate the addition of other languages. The block colors have been selected to ensure that the task can be completed by people with most forms of colorblindness, and the trials were generated to cover puzzles with varying levels of visual and structural complexity. The task also works with a mouse or touchscreen either online or offline, such that it can be used with multiple devices (computer/laptop, tablet,

smartphone) in various settings (laboratory, clinic, remote). Increased impact can be achieved by incorporating engaging and user-friendly interfaces to maintain participant interest and motivation, providing clear and concise instructions, supplemented with visual and audio aids, and a practice session to ensure that participants understand the task. The task should be made openly available to other researchers by sharing it on open platforms, such as the Open Science Framework (OSF) and GitHub/GitLab. Our task is housed in GitLab and linked to the OSF, which allowed us to generate a DOI. As an open-source project, this will encourage collaboration and contributions from the wider research community, thereby increasing its impact.

Developing the task and procedurally generated trials in parallel

The task itself should be developed following the design mockups alongside the procedurally generated trials. This will allow for a better understanding of how the trial data should be formatted and what data should be output from the generated trials. For example, our task was designed in PsychoPy/PsychoJS, which uses .csv files to define the trials. When designing the task, we also used an invisible grid, noting the x - and y -axis positions for the placement of blocks. Thus, our procedural generation algorithm outputs those positions in a

Algorithm 1: Breadth-First Search

Data: *initialState*: starting block positions and colors on the grid
goalState: an empty grid (all blocks cleared)
Result: The shortest path to solve the puzzle.

begin

```

  Initialize an empty queue Q;
  Enqueue(Q, initialState); /* add initialState to Q */
  while Q is not empty do
    currentState ← Dequeue(Q); /* remove the state at the front of the
      queue and assign it to currentState for exploration */
    Iterate over blocks in currentState;
    for each block do
      Perform all possible moves with the block;
      for all possible moves do
        newState ← Apply gravity and elimination rules;
        newState.history ← Save history that led to newState;
        if newState==goalState then
          return newState;
        else
          Enqueue(Q, newState);
        end
      end
    end
  end
  return "No solution found";
end

```

Fig. 8 Pseudocode of the breadth-first search algorithm. Note. The shortest path refers to the sequence of moves that solves the puzzle in the minimum number of steps. Upon completion, the algorithm pro-

vides the final configuration, which was an empty grid, along with the history of previous configurations that led to this state

.csv format for simple integration into the task. Developing the task in parallel will also help troubleshoot any practical issues that could impact trial generation, such as technical limitations that require rule changes.

Procedural generation guidelines for cognitive assessment

Selecting appropriate algorithms

We opted to pseudo-randomize trial generation following a specific set of rules, with resulting trials validated through

the BFS algorithm. Based on previous puzzle generation algorithms reviewed by De Kegel and Haahr (2019), this was an appropriate solution for our task design. However, different cognitive tasks may be better suited to other generative algorithms, and these should be fully considered prior to moving forward.

Developing procedural generation rules

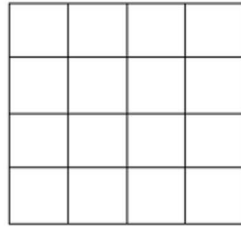
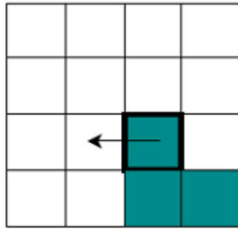
Constraints (i.e., task-relevant rules the algorithm must follow) can be added to guide the procedural generation of new puzzles, making sure each puzzle meets the task

Solvable in 1 move

MOVE 1:

Perform: move_left at position: (1, 2)

FINAL STATE



Solvable in 2 moves

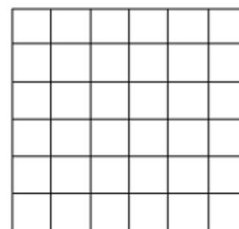
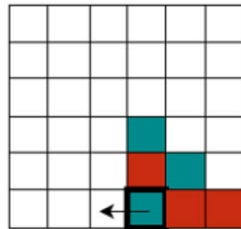
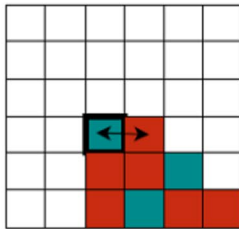
MOVE 1:

Perform: exchange_right at position: (2, 2)

MOVE 2:

Perform: move_left at position: (0, 3)

FINAL STATE



Solvable in 3 moves

MOVE 1:

Perform: moving_right at position: (1, 1)

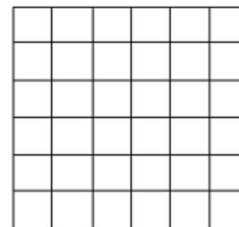
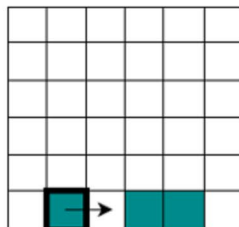
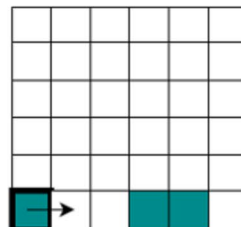
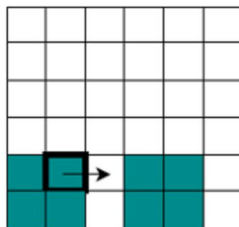
MOVE 2:

Perform: moving_right at position: (0, 0)

MOVE 2:

Perform: moving_right at position: (0, 0)

FINAL STATE



Solvable in 4 moves

MOVE 1:

Perform: move_right at position: (2, 3):

MOVE 2:

Perform: exchange_right at position: (2, 2):

MOVE 3:

Perform: move_left at position: (1, 3):

MOVE 4:

Perform: move_left at position: (0, 4):

FINAL STATE

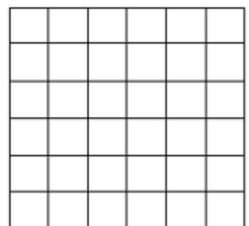
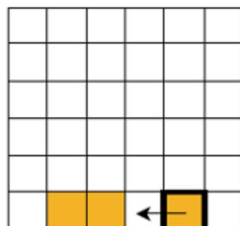
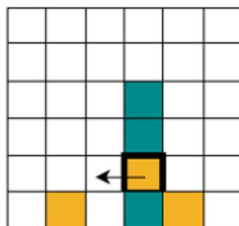
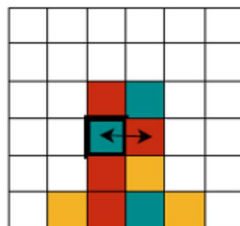
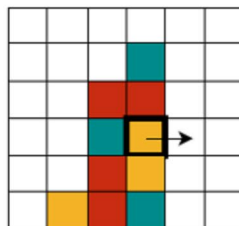


Fig. 9 Example of procedurally generated batch of puzzles by number of moves to solve

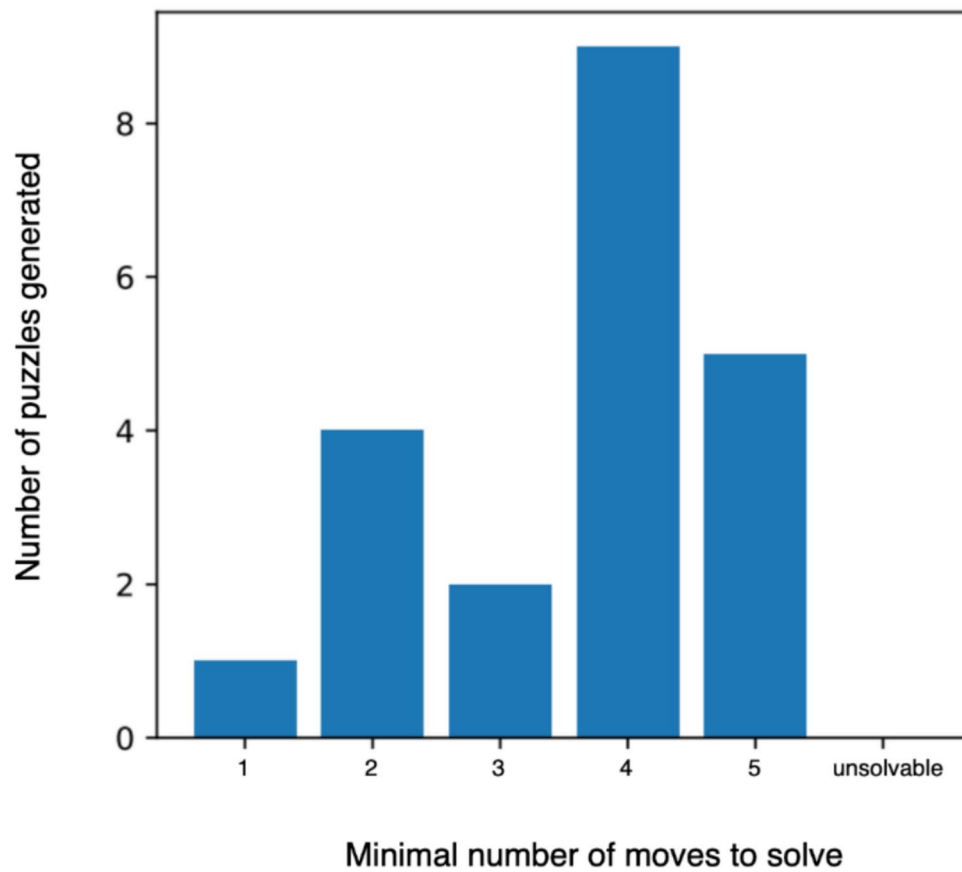


Fig. 10 Statistical data of puzzles from the procedurally generated batch: a bar chart of puzzle counts and moves required for solvability

Table 3 Batch generation example: puzzles with 3 to 12 blocks, up to 3 colors, 4×4 to 12×6 grid size, and solvable in 5 or fewer moves, consistently generated without failures

Number of blocks	Number of colors	Grid size	Minimum moves
3	1	4×4	1
[4, 6]	1	$[5 \times 5, 7 \times 7]$	2
[7, 8]	1	$[8 \times 8, 9 \times 9]$	3
[9, 10, 12]	1	$[10 \times 10, 11 \times 11, 13 \times 13]$	4
[5, 11]	1	$[6 \times 6, 12 \times 12]$	5
[6, 9]	2	12×6	2
[7, 8, 12]	2	12×6	4
[10, 11]	2	12×6	5
[10, 11, 12]	3	12×6	4
9	3	12×6	5

For each color, we grouped the data by the number of moves and indicated the corresponding blocks and grid sizes using square brackets. We adopted this Python list notation for clarity

Table 4 Batch generation example: puzzles with 3 to 12 blocks, up to 3 colors, 16×16 fixed grid size, and solvable in 5 or fewer moves, generated with failures

Number of blocks	Number of colors	Grid size	Minimum moves	Generated/Failed
3	1	16×16	1	Generated
[4, 6, 7]	1	16×16	2	Generated
[5, 8, 9, 10]	1	16×16	3	Generated
[11, 12, 13, 14]	1	16×16	4	Generated
15	1	16×16	5	Generated
6	2	16×16	2	Generated
[7, 9]	2	16×16	3	Generated
[10, 12]	2	16×16	4	Generated
[8, 11, 13, 14, 15]	2	16×16	5	Generated
9, 11	3	16×16	3	Generated
14	3	16×16	5	Generated
[10, 12, 13, 15]	3	16×16	-	Failed

For each color, we grouped the data by the number of moves and indicated the corresponding blocks and grid sizes using square brackets. We adopted this Python list notation for clarity

requirements, and improving the efficiency of trial generation. It is particularly important with randomized algorithms to consider constraints that maximize the chances of creating a solvable puzzle while minimizing the time the algorithm spent generating it. For example, we specified that blocks could only be placed in valid positions (bottom row or on top of another block), could not be placed where it would lead to three or more of the same color aligned together, and should be placed with no more than one empty cell between them. We also added a stacking probability to vary the type of trials generated. These constraints ensured valid pseudo-random placement and fewer failed generations. Such rules should be carefully considered before implementing the trial generation algorithm.

Generation of trials offline, review, and validation

Procedural trial generation is a resource-intensive process and can involve a degree of randomness. While it would be possible to generate trials in real time, this could affect performance and user experience. It would also mean that trials could not be validated, potentially reducing the validity of the task. Finally, real-time generation would also likely create different trials across participants, which could undermine analyses comparing performance between individuals. Thus, we recommend generating trials offline, reviewing them internally and adapting the rules or algorithms, if necessary. Once a candidate set of trials has been developed, these should be psychometrically validated against the corresponding gold-standard cognitive measure.

Here, we have outlined the algorithmic validation of procedural content generation for cognitive tasks as well as the design, development, and implementation of the task itself, with validation of the trials reserved for future work. This scope allows for a thorough examination of the algorithmic approach to promote its use in broader contexts and was necessary given the ongoing nature of our validation study. Importantly, the current task iteration has not yet been psychometrically validated, and any conclusions drawn from its use should therefore be interpreted with caution. Our validation plan involves (1) convergent validity through comparison with gold-standard measures of planning (e.g., *Stockings of Cambridge*), (2) divergent validity in terms of other cognitive domains (e.g., memory), (3) internal consistency and 2-week test–retest reliability, and (4) sensitivity/specificity with regard to identifying cognitive deficits in psychiatric samples.

Future directions

One limitation of our current puzzle generation method is the lack of precise control over difficulty. To address this limitation, it would first be appropriate to define what

constitutes difficulty in the context of our task. While parameters such as the number of moves, blocks, or colors could be used as proxies, they do not necessarily reflect the level of difficulty (e.g., puzzles with more moves can sometimes be more straightforward to solve than shorter but more deceptive ones). Additional parameters could therefore be considered, such as look-ahead depth (i.e., how many future moves need to be planned to make a decision on the best sequence; Rosenberg & Kestner, 2008); branching factor (i.e., how many choices are available after each move; Lantz et al., 2017); deceptive attractors/sub-goals (i.e., moves that seem helpful but delay progress; Yang, 2004); grid density (i.e., how crowded the board is; Lantz et al., 2017); and color entropy (i.e., how varied and evenly distributed the colors are; Qu et al., 2022).

These features can influence cognitive load in different ways (Callaway et al., 2022; Wiener et al., 2009). For example, deeper look-ahead can increase demands on planning directly as well as working memory. A higher branching factor may increase decision-making complexity, and deceptive sub-goals could increase the chance of errors. In addition, grid density and color entropy would be expected to increase demands on visual perception, spatial attention, and working memory. These parameters can be automatically estimated during the generation of new puzzles. For example, look-ahead depth could be approximated by how many steps an algorithm needs to simulate to identify the next best move. Branching factor could be computed by counting the number of valid moves available after each action. Deceptive sub-goals could be flagged when valid moves lead away from the optimal solution. Grid density and color entropy could be calculated from the puzzle layout. Logging these parameters would enable application of psychometric models—such as item response theory (Wilson, 2023)—to explore how specific puzzle characteristics relate to player performance and cognitive demand.

The above approach focuses on theorizing puzzle difficulty based on structural features expected to influence cognitive load. A complementary strategy is to infer difficulty empirically, using observed player performance. Since this approach relies on empirical data, the difficulty of each puzzle can be validated and used to construct a calibrated item bank suitable for real-time difficulty adjustment in our executive function task. This process would involve (1) generating a large set of puzzles offline; (2) validating these puzzles with a normative sample to collect performance data—such as average time to completion, number of attempts, or error rates—and labeling each puzzle with an estimated difficulty level based on normative performance; and (3) using these labeled puzzles during the administration of the task, where they would be dynamically selected based on the current participant's performance.

By empirically validating puzzles in advance and avoiding the generation of entirely new puzzles during task administration, this approach helps preserve the psychometric integrity and reproducibility of the assessment. Future implementation of this adaptive difficulty feature would require (1) tracking individual performance metrics in real time (e.g., whether the puzzle was solved, time taken, whether the solution was optimal, number of correct responses in a row) and (2) implementing an adaptive algorithm using these indicators to select subsequent puzzles of appropriate difficulty. For instance, based on how quickly a participant solves puzzles and how many they complete consecutively, the algorithm could adjust the rate at which the difficulty increases or decreases.

If real-time puzzle generation is desired, exploring algorithms inspired by natural evolution—such as genetic algorithms—could provide effective approaches. Unlike our current offline method, which generates puzzles in a random manner, genetic algorithms work by applying small variations to a collection of acceptable puzzles and gradually refining them over time. This process can help generate solvable puzzles tailored to specific criteria such as difficulty and variety (Bukhori et al., 2024; Volden et al., 2023). It would be of interest to explore the use of genetic algorithms to dynamically personalize the task based on real-time participant performance. Finally, investigating the application of this tool in clinical settings is warranted to assess its utility in diagnosing and monitoring cognitive impairments.

Conclusion

We have outlined the development of a novel executive function planning task using a computational procedure to automatically generate cognitive trials. Procedural trial generation represents an exciting new direction in digital cognitive assessment, with the potential to facilitate large-scale task development and repeated assessments in both research (e.g., cognitive variability) and clinical (e.g., cognitive monitoring) settings. By providing a detailed framework and guidelines for procedural trial generation for cognitive assessment, we aim to facilitate the development of other digital cognitive tasks in the future. Our collaborative approach makes it a valuable resource for anyone wishing to develop projects collaboratively and enhance the development of innovative solutions in digital cognitive assessment. The automatic generation of trials represents an important step towards modernizing cognitive assessment methods, potentially opening new avenues for studying the dynamic nature of cognition in real-life contexts using repeated assessments.

Acknowledgements We would like to thank Jiaxuan Deng for her help with designing the task mockups. We acknowledge the use of

high-performance computing resources as part of the Digital Research Alliance of Canada.

Authors' contributions Conceptualization: KML, CD, ET, DRC, GS; Methodology: KML, CD, KS; Software: CD, KML, AC; Validation: CD, KML, ET, DRC, GS; Formal Analysis: CD, AC; Investigation: KML, CD, AC, KS; Resources: KML; Data Curation: CD, AC; Writing—Original Draft: CD, KML, AC, KS; Writing Review & Editing: CD, KML, AC, KS, ET, DRC, GS; Visualization: CD, AC, KS; Supervision: KML; Project Administration: KML; Funding Acquisition: KML.

Funding This research was undertaken thanks in part to funding from the Canada First Research Excellence Fund, awarded through the Healthy Brains, Healthy Lives initiative at McGill University (2b-NISU-22), and from the Natural Sciences and Engineering Research Council (RGPIN-2024-06378 & DGEGR-2024-00135).

Declarations

Ethics approval Not applicable.

Consent to participate Not applicable.

Consent for publication Not applicable.

Conflicts of interest/Competing interests The authors declare that they have no conflict of interest or competing interests.

Open practices/Data availability statement Sample puzzles generated via our procedural script (<https://doi.org/10.17605/OSF.IO/HNAQ9>), along with the sheet used to document issues (functionality, usability, design) during usability and functional testing of our task (<https://doi.org/10.17605/OSF.IO/3PZ74>), are openly accessible via the Open Science Framework. Our Executive Function planning task code (<https://doi.org/10.17605/OSF.IO/3PZ74>) and our procedural puzzle generation package (<https://doi.org/10.17605/OSF.IO/HNAQ9>) are openly available via the Open Science Framework.

References

- Abramovitch, A., Short, T., & Schweiger, A. (2021). The C Factor: Cognitive dysfunction as a transdiagnostic dimension in psychopathology. *Clinical Psychology Review*, 86, Article 102007. <https://doi.org/10.1016/j.cpr.2021.102007>
- Baniqued, P. L., Lee, H., Voss, M. W., Basak, C., Cosman, J. D., Desouza, S., Severson, J., Salthouse, T. A., & Kramer, A. F. (2013). Selling points: What cognitive abilities are tapped by casual video games? *Acta Psychologica (Amsterdam)*, 142(1), 74–86. <https://doi.org/10.1016/j.actpsy.2012.11.009>
- Bridges, D., Pitiot, A., MacAskill, M. R., & Peirce, J. W. (2020). The timing mega-study: Comparing a range of experiment generators, both lab-based and online. *PeerJ*, 8, Article e9414. <https://doi.org/10.7717/peerj.9414>
- Bukhori, I., Felix, J., & Ali, S. (2024). Using genetic algorithm to solve puzzle games: A review. *Journal of Computer Networks, Architecture and High Performance Computing*, 6(1), 201–211. <https://doi.org/10.47709/cnahpc.v6i1.3348>
- Bundy, A., & Wallen, L. (1984). Breadth-First Search. In A. Bundy & L. Wallen (Eds.), *Catalogue of Artificial Intelligence Tools* (pp. 13–13). Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-642-96868-6_25

- Calamia, M., Markon, K., & Tranel, D. (2012). Scoring higher the second time around: Meta-analyses of practice effects in neuropsychological assessment. *The Clinical Neuropsychologist*, 26(4), 543–570. <https://doi.org/10.1080/13854046.2012.680913>
- Caldwell, B., Cooper, M., Reid, L. G., Vanderheiden, G., Chisholm, W., Slatin, J., & White, J. (2008). Web content accessibility guidelines (WCAG) 2.0. *WWW Consortium (W3C)*, 290(1–34), 5–12.
- Callaway, F., van Opheusden, B., Gul, S., Das, P., Krueger, P. M., Griffiths, T. L., & Lieder, F. (2022). Rational use of cognitive resources in human planning. *Nature Human Behaviour*, 6(8), 1112–1125.
- Chen, L., Zhen, W., & Peng, D. (2023). Research on digital tool in cognitive assessment: A bibliometric analysis. *Frontiers in Psychiatry*, 14, Article 1227261. <https://doi.org/10.3389/fpsy.2023.1227261>
- De Kegel, B., & Haahr, M. (2019). Procedural puzzle generation: A survey. *IEEE Transactions on Games*, 12(1), 21–40.
- Diamond, A. (2013). Executive functions. *Annual Review of Psychology*, 64, 135–168. <https://doi.org/10.1146/annurev-psych-113011-143750>
- Goldberg, T. E., Keefe, R. S., Goldman, R. S., Robinson, D. G., & Harvey, P. D. (2010). Circumstances under which practice does not make perfect: A review of the practice effect literature in schizophrenia and its relevance to clinical treatment studies. *Neuropsychopharmacology*, 35(5), 1053–1062. <https://doi.org/10.1038/npp.2009.211>
- Guimond, S., Keshavan, M. S., & Torous, J. B. (2019). Towards remote digital phenotyping of cognition in schizophrenia. *Schizophrenia Research*, 208, 36–38. <https://doi.org/10.1016/j.schres.2019.04.016>
- Harvey, P. D. (2019). Domains of cognition and their assessment. *Dialogues in Clinical Neuroscience*, 21(3), 227–237. <https://doi.org/10.31887/DCNS.2019.21.3/pharvey>
- Hays, R., Henson, P., Wisniewski, H., Hendel, V., Vaidyam, A., & Torous, J. (2019). Assessing cognition outside of the clinic: Smartphones and sensors for cognitive assessment across diverse psychiatric disorders. *Psychiatric Clinics of North America*, 42(4), 611–625. <https://doi.org/10.1016/j.psc.2019.08.003>
- Intelligent Systems. (1995). Panel De Pon. In *[Video Game]*: Nintendo.
- Iverson, G. L., Brooks, B. L., White, T., & Stern, R. A. (2008). Neuropsychological Assessment Battery: Introduction and advanced interpretation. In A. M. Horton Jr & D. Wedding (Eds.), *The neuropsychology handbook* (3rd ed., pp. 279–343). Springer.
- Jutten, R. J., Grandoit, E., Foldi, N. S., Sikkes, S. A. M., Jones, R. N., Choi, S. E., Lamar, M. L., Loudon, D. K. N., Rich, J., Tommet, D., Crane, P. K., & Rabin, L. A. (2020). Lower practice effects as a marker of cognitive performance and dementia risk: A literature review. *Alzheimer's & Dementia: Translational Research & Clinical Interventions*, 12(1), Article e12055. <https://doi.org/10.1002/dad2.12055>
- Kiely, K. M. (2014). Cognitive Function. In A. C. Michalos (Ed.), *Encyclopedia of Quality of Life and Well-Being Research* (pp. 974–978). Springer Netherlands. https://doi.org/10.1007/978-94-007-0753-5_426
- Kilciksiz, C. M., Keefe, R., Benoit, J., Ongur, D., & Torous, J. (2020). Verbal memory measurement towards digital perspectives in first-episode psychosis: A review. *Schizophrenia Research: Cognition*, 21, Article 100177. <https://doi.org/10.1016/j.scog.2020.100177>
- Korn, O., Blatz, M., Rees, A., Schaal, J., Schwind, V., & Görlich, D. (2017). Procedural content generation for game props? A study on the effects on user experience. *Computers in Entertainment*, 15(2), 1–15. <https://doi.org/10.1145/2974026>
- Lantz, F., Isaksen, A., Jaffe, A., Nealen, A., & Togelius, J. (2017). Depth in Strategic Games. AAAI Workshops,
- Lavigne, K. M., Raucher-Chéné, D., Sauvé, G., & Thibaut, É. (2025). Open Cognitive Battery. 10.17605/OSF.IO/PKSZJ
- Lavigne, K. M., Sauve, G., Raucher-Chene, D., Guimond, S., Lecomte, T., Bowie, C. R., Menon, M., Lal, S., Woodward, T. S., Bodnar, M. D., & Lepage, M. (2022). Remote cognitive assessment in severe mental illness: A scoping review. *Schizophrenia*, 8(1), Article 14. <https://doi.org/10.1038/s41537-022-00219-x>
- Llado, A., Froelich, L., Khandker, R. K., Roset, M., Black, C. M., Lara, N., Chekani, F., & Ambegaonkar, B. M. (2021). Assessing the progression of Alzheimer's Disease in real-world settings in three European countries. *Journal of Alzheimer's Disease*, 80(2), 749–759. <https://doi.org/10.3233/JAD-201172>
- Maruff, P., Thomas, E., Cysique, L., Brew, B., Collie, A., Snyder, P., & Pietrzak, R. H. (2009). Validity of the CogState brief battery: Relationship to standardized tests and sensitivity to cognitive impairment in mild traumatic brain injury, schizophrenia, and AIDS dementia complex. *Archives of Clinical Neuropsychology*, 24(2), 165–178. <https://doi.org/10.1093/arclin/acp010>
- McKinney, T. L., Euler, M. J., & Butner, J. E. (2020). It's about time: The role of temporal variability in improving assessment of executive functioning. *The Clinical Neuropsychologist*, 34(4), 619–642. <https://doi.org/10.1080/13854046.2019.1704434>
- Millan, M. J., Agid, Y., Brune, M., Bullmore, E. T., Carter, C. S., Clayton, N. S., Connor, R., Davis, S., Deakin, B., DeRubeis, R. J., Dubois, B., Geyer, M. A., Goodwin, G. M., Gorwood, P., Jay, T. M., Joels, M., Mansuy, I. M., Meyer-Lindenberg, A., Murphy, D., ... Young, L. J. (2012). Cognitive dysfunction in psychiatric disorders: Characteristics, causes and the quest for improved therapy. *Nature Reviews. Drug Discovery*, 11(2), 141–168. <https://doi.org/10.1038/nrd3628>
- Nielsen, J. (1994). *Usability engineering*. Morgan Kaufmann.
- Norman, D. (2013). *The design of everyday things: Revised and expanded edition*. Basic books.
- O'Brien, H. L., & Toms, E. G. (2008). What is user engagement? A conceptual framework for defining user engagement with technology. *Journal of the American Society for Information Science and Technology*, 59(6), 938–955.
- Peirce, J., Gray, J. R., Simpson, S., MacAskill, M., Hochenberger, R., Sogo, H., Kastman, E., & Lindelov, J. K. (2019). PsychoPy2: Experiments in behavior made easy. *Behavior Research Methods*, 51(1), 195–203. <https://doi.org/10.3758/s13428-018-01193-y>
- Persson, H., Åhman, H., Yngling, A. A., & Gulliksen, J. (2015). Universal design, inclusive design, accessible design, design for all: Different concepts—one goal? On the concept of accessibility—Historical, methodological and philosophical aspects. *Universal Access in the Information Society*, 14, 505–526.
- Pietrzak, R. H., Olver, J., Norman, T., Piskulic, D., Maruff, P., & Snyder, P. J. (2009). A comparison of the CogState Schizophrenia Battery and the Measurement and Treatment Research to Improve Cognition in Schizophrenia (MATRICS) Battery in assessing cognitive impairment in chronic schizophrenia. *Journal of Clinical and Experimental Neuropsychology*, 31(7), 848–859. <https://doi.org/10.1080/13803390802592458>
- Qu, C., DeWind, N. K., & Brannon, E. M. (2022). Increasing entropy reduces perceived numerosity throughout the lifespan. *Cognition*, 225, Article 105096. <https://doi.org/10.1016/j.cognition.2022.105096>
- Robbins, T. W., James, M., Owen, A. M., Sahakian, B. J., McInnes, L., & Rabbitt, P. (1994). Cambridge Neuropsychological Test Automated Battery (CANTAB): A factor analytic study of a large sample of normal elderly volunteers. *Dementia*, 5(5), 266–281. <https://doi.org/10.1159/000106735>
- Rosenberg, R. S., & Kestner, J. (2008). Look-ahead and one-person games. *Journal of Cybernetics*, 2(4), 27–42. <https://doi.org/10.1080/01969727208545856>
- Scharfen, J., Peters, J. M., & Holling, H. (2018). Retest effects in cognitive ability tests: A meta-analysis. *Intelligence*, 67, 44–66.

- Shallice, T. (1982). Specific impairments of planning. *Philosophical Transactions of the Royal Society of London. Series b, Biological Sciences*, 298(1089), 199–209. <https://doi.org/10.1098/rstb.1982.0082>
- Smith, G. (2014). *Understanding procedural content generation* Proceedings of the SIGCHI Conference on Human Factors in Computing Systems, Toronto, Ontario, Canada. <https://doi.org/10.1145/2556288.2557341>
- Togelius, J., Yannakakis, G. N., Stanley, K. O., & Browne, C. (2011). Search-based procedural content generation: A taxonomy and survey. *IEEE Transactions on Computational Intelligence and AI in Games*, 3(3), 172–186. <https://doi.org/10.1109/tciaig.2011.2148116>
- Vita, A., Barlati, S., Ceraso, A., Nibbio, G., Ariu, C., Deste, G., & Wykes, T. (2021). Effectiveness, core elements, and moderators of response of cognitive remediation for schizophrenia: A systematic review and meta-analysis of randomized clinical trials. *JAMA Psychiatry*, 78(8), 848–858. <https://doi.org/10.1001/jamapsychiatry.2021.0620>
- Volden, T., Grbic, D., & Burelli, P. (2023). Procedurally generating rules to adapt difficulty for narrative puzzle games. 2023 IEEE Conference on Games (CoG),
- Weintraub, S., Dikmen, S. S., Heaton, R. K., Tulsky, D. S., Zelazo, P. D., Bauer, P. J., Carlozzi, N. E., Slotkin, J., Blitz, D., Wallner-Allen, K., Fox, N. A., Beaumont, J. L., Mungas, D., Nowinski, C. J., Richler, J., Deocampo, J. A., Anderson, J. E., Manly, J. J., Borosh, B., ... Gershon, R. C. (2013). Cognition assessment using the NIH Toolbox. *Neurology*, 80(11 Suppl 3), S54–64. <https://doi.org/10.1212/WNL.0b013e3182872ded>
- Wiener, J. M., Ehbauer, N. N., & Mallot, H. A. (2009). Planning paths to multiple targets: Memory involvement and planning heuristics in spatial problem solving. *Psychological Research Psychologische Forschung*, 73(5), 644–658. <https://doi.org/10.1007/s00426-008-0181-3>
- Wilson, M. (2023). *Constructing measures: An item response modeling approach*. Routledge.
- Yang, S. (2004). Adaptive group mutation for tackling deception in genetic search.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.